Simple Flower Classifier using an Artificial Neural Network

Artificial Neural Networks - Activity 1 Report

Team members
Kirish - RA2411056050002 - B.Tech CSE-DS
[Teammate Name] - [Teammate Reg No] - B.Tech CSE-DS

Tools: Python, TensorFlow/Keras, scikit-learn, Matplotlib, Streamlit
GitHub: [paste repo link] | **Live app:** [paste app link]

Abstract

This project builds and deploys a simple flower classifier using a feed-forward artificial neural network. The network is trained on the Iris dataset to predict a flower's species (setosa, versicolor or virginica) from four measurements. The model reached 96.7% accuracy on unseen test data and was deployed as an interactive Streamlit web application.

1. Introduction

Classification is a core machine-learning task: assigning an input to one of several categories. Artificial neural networks (ANNs) are inspired by the brain and consist of layers of connected neurons. Each neuron computes a weighted sum of its inputs, adds a bias and applies a non-linear activation function. By stacking layers, an ANN can learn complex relationships between inputs and outputs.

The objective of this activity is to design, train, evaluate and deploy a simple flower classifier. The Iris flower problem is a standard benchmark for this purpose: it is small enough to train in seconds, yet it demonstrates the full workflow of data preparation, modelling, evaluation and deployment.

2. Dataset

The Iris dataset (Fisher, 1936; distributed with scikit-learn) contains 150 samples, 50 for each of three species. Each sample has four numeric features in centimetres: sepal length, sepal width, petal length and petal width. The dataset has no missing values and is perfectly balanced. Petal measurements separate the species most clearly, as seen in the pair plot below: setosa is linearly separable from the others, while versicolor and virginica overlap slightly.

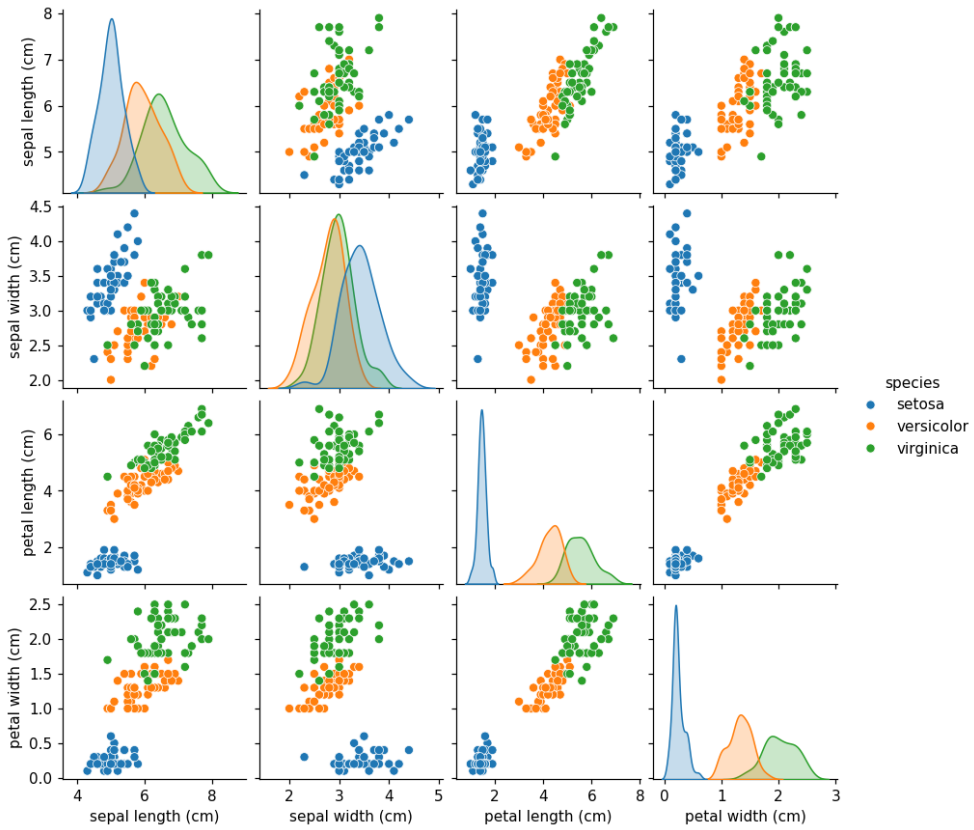


Figure 1: Pair plot of the four features coloured by species.

3. Methodology

3.1 Preprocessing. The data was split into 80% training (120 samples) and 20% testing (30 samples) using a stratified split so each species is equally represented. Features were standardised with StandardScaler, fitted on the training set only to avoid data leakage. 20% of the training data was held out as a validation set during training.

3.2 Model architecture. A fully connected network with two hidden layers was used. ReLU activations in the hidden layers introduce non-linearity, and the softmax output layer converts the three outputs into class probabilities that sum to one.

Layer	Type	Units	Activation	Parameters
Input	-	4	-	0
Hidden 1	Dense	16	ReLU	80
Hidden 2	Dense	8	ReLU	136
Output	Dense	3	Softmax	27
Total				243

3.3 Training configuration. The model was compiled with the Adam optimiser (learning rate 0.01) and sparse categorical cross-entropy loss, which suits integer class labels. It was trained for 100 epochs with a batch size of 16. Weights are updated by backpropagation: the loss is computed in a forward pass and its gradients are propagated backwards to adjust each weight.

3.4 Evaluation. Performance was measured on the untouched test set using accuracy, precision, recall, F1-score and a confusion matrix.

4. Results

Training converged quickly: the loss fell steadily and accuracy rose within the first few dozen epochs, reaching 100.0% on the training data and 100.0% on the validation data. The training and validation curves stay close together, which indicates the model generalises well rather than memorising the data.

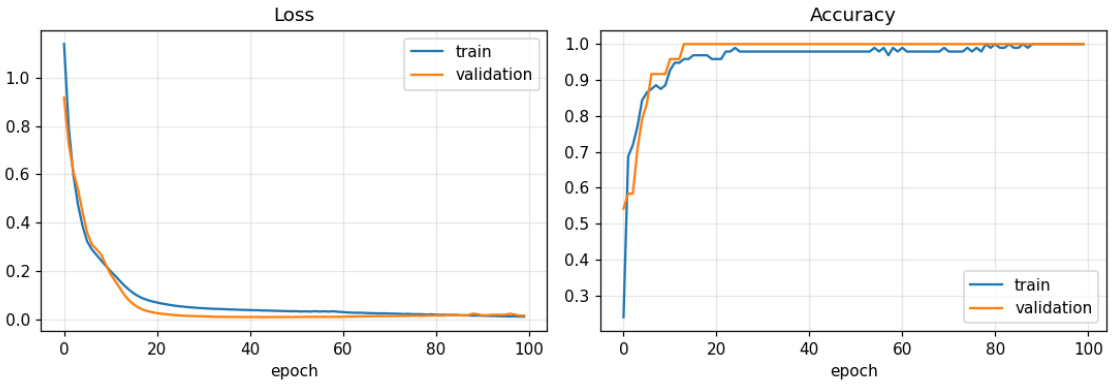


Figure 2: Training and validation loss and accuracy over 100 epochs.

On the 30 unseen test samples the network achieved **96.67% accuracy**. Setosa was classified perfectly, and the only error was one versicolor flower predicted as virginica, which is expected because these two species overlap in feature space.

Class	Precision	Recall	F1-score	Support
Setosa	1.00	1.00	1.00	10
Versicolor	1.00	0.90	0.95	10
Virginica	0.91	1.00	0.95	10

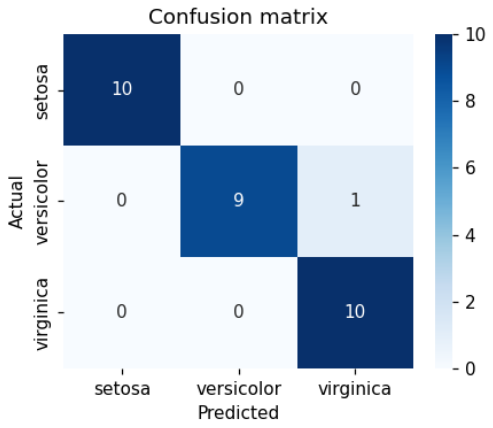


Figure 3: Confusion matrix on the test set.

5. Deployment

The trained model was saved in Keras format (*iris_model.keras*) together with the fitted scaler (*scaler.pkl*). A Streamlit application (*app.py*) loads both files, lets the user set the four measurements with sliders, applies the same scaling used in training and displays the predicted species, the confidence and a bar chart of the class probabilities. The code was pushed to GitHub and hosted on Streamlit Community Cloud, with dependencies listed in *requirements.txt*, so the classifier can be used from any browser.

Sample predictions. Measurements of 5.1, 3.5, 1.4, 0.2 cm are classified as setosa; 6.7, 3.0, 5.2, 2.3 cm as virginica, matching the known labels.

6. Discussion and Limitations

The small dataset means the test score rests on only 30 samples, so a single misclassification changes accuracy by about 3.3 percentage points. Cross-validation would give a more reliable estimate. The model also only knows the three Iris species and works on measurements, not photographs. A larger image-based flower classifier would need a convolutional neural network and a bigger dataset.

7. Conclusion

A compact ANN with two hidden layers classified Iris flowers with 96.7% test accuracy and was successfully deployed as a web application. The project covered the full machine-learning pipeline: data exploration, preprocessing, model design, training, evaluation and deployment.

References

1. Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. Annals of Eugenics.
2. Chollet, F. et al. Keras / TensorFlow documentation, tensorflow.org.
3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.
4. Streamlit documentation, docs.streamlit.io.